МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 5

Тема: Структурные паттерны: Компоновщик, Декоратор

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы структурных паттернов проектирования «Фасад» (Facade) и «Приспособленец» (Flyweight) и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать два структурных паттерна из списка: «Фасад», «Приспособленец».

В соответствии с заданием были выбраны паттерны «Фасад» (Facade) и «Приспособленец» (Flyweight).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, а также демонстрирует работу структурных паттернов: фасад скрывает сложность работы с подсистемой пароля, а приспособленец позволяет разделять объекты стиля между всеми ячейками сетки.

1. Описание паттерна «Фасад»

Паттерн «Фасад» (Facade) - это структурный паттерн проектирования, который предоставляет простой интерфейс к сложной системе классов, библиотеке или фреймворку. Фасад скрывает внутреннюю сложность подсистемы и делает её использование более удобным для клиентского кода.

Данный паттерн применяется в ситуациях, когда нужно упростить работу с подсистемой, уменьшить число зависимостей между клиентом и подсистемой, или разбить систему на слои.

В разработанном приложении паттерн «Фасад» используется для упрощения работы с подсистемой графического пароля. В программе определён класс PasswordFacade, который скрывает:

- PasswordManager - Singleton, отвечающий за хэширование и работу с файлом graphical_password.json;
- список шаблонов PasswordPattern;
- валидацию последовательности через функцию validate_sequence();
- операции создания, проверки и сброса пароля.

UI-код вызывает только методы фасада:

result, message = self.facade.create_password(self.selected)

result, message = self.facade.verify(self.selected)

self.facade.reset()

self.facade.has_password()

Таким образом, класс AppWindow не знает о внутреннем устройстве подсистемы и работает только через единый простой интерфейс.

2. Описание паттерна «Приспособленец»

Паттерн «Приспособленец» (Flyweight) - это структурный паттерн проектирования, который позволяет экономить память за счёт разделения общего состояния между множеством мелких объектов. Вместо создания отдельного объекта для каждого элемента программа использует общий (разделяемый) объект, если состояние повторяется.

Данный паттерн применяется в ситуациях, когда требуется создать большое количество однотипных объектов, и большая часть их состояния может быть вынесена наружу.

В разработанном приложении паттерн «Приспособленец» используется для экономии памяти при отрисовке ячеек сетки. В программе определены следующие роли:

CellStyle - приспособленец, содержащий общее состояние стиля (цвет фона, цвет текста, шрифт, радиус);

CellStyleFactory - фабрика с кэшем, которая возвращает один и тот же объект для одинаковых стилей;

AppWindow - клиент, использующий приспособленцев.

Все 9 ячеек сетки используют два общих объекта CellStyle: "normal" (обычная ячейка) и "selected" (выбранная ячейка). Вместо 9 отдельных объектов стиля создаётся только 2, и они разделяются между всеми ячейками.

Кнопка «Статистика Flyweight» демонстрирует экономию: показывает количество ячеек (9) и количество уникальных объектов CellStyle (2).

3. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter.

Основное окно содержит сетку из девяти скруглённых ячеек, пронумерованных от 1 до 9. Пользователь формирует пароль путём последовательного выбора ячеек.

К паролю предъявляются требования:

- длина последовательности от 4 до 9 ячеек;
- одна и та же ячейка не может быть выбрана повторно;
- созданный пароль сохраняется в виде SHA-256 хэша.

Приложение содержит два режима: «Создание пароля» и «Вход».

Также в приложении реализованы:

- панель шаблонов («Крест», «Квадрат», «Зигзаг»);
- кнопка «Статистика Flyweight» для демонстрации разделения объектов стиля;
- кнопка «Сбросить сохранённый пароль»;
- кнопка «Очистить» для сброса текущей последовательности.

Все действия с паролем выполняются через PasswordFacade.

4. Диаграмма классов

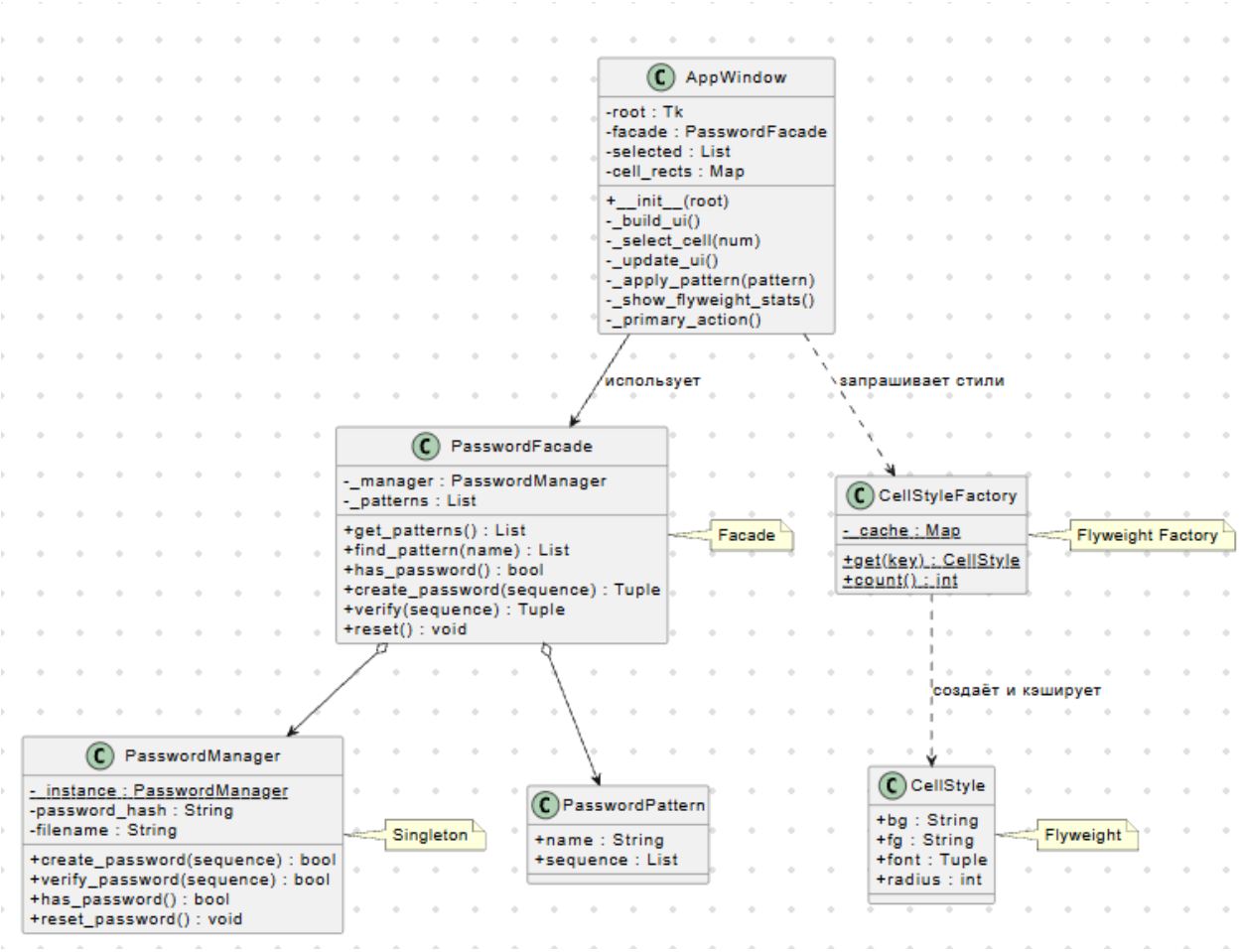


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит:

– PasswordFacade - фасад над подсистемой пароля;
– PasswordManager - Singleton с хэшированием (из лабы 1);
– PasswordPattern - шаблон пароля;
– CellStyle - приспособленец;
– CellStyleFactory - фабрика с кэшем;
– AppWindow - основной класс интерфейса.

5. Реализация паттерна «Фасад»

Основная часть реализации паттерна находится в классе PasswordFacade.

```
class PasswordFacade:
    def __init__(self):
        self._manager = PasswordManager()
```

```
        self._patterns = [
            PasswordPattern("Крест", [2, 5, 8, 4, 6]),
            PasswordPattern("Квадрат", [1, 2, 3, 6, 9, 8, 7, 4]),
            PasswordPattern("Зигзаг", [1, 2, 4, 3, 5, 7, 6, 8, 9]),
        ]

    def create_password(self, sequence):
        if not validate_sequence(sequence):
            return False, "Некорректная последовательность."
        if self._manager.has_password():
            return None, "Пароль уже существует"
        self._manager.create_password(sequence)
        return True, "Графический пароль успешно создан."

    def verify(self, sequence):
        if not self._manager.has_password():
            return False, "Пароль не создан."
        if not validate_sequence(sequence):
            return False, "Некорректная последовательность."
        if self._manager.verify_password(sequence):
            return True, "Пароль верный. Доступ разрешён."
        return False, "Пароль неверный. Доступ запрещён."

    def reset(self):
        self._manager.reset_password()

    def has_password(self):
        return self._manager.has_password()

    def get_patterns(self):
```

```
        return list(self._patterns)
```

UI вызывает эти методы вместо того, чтобы работать с PasswordManager и validate_sequence() напрямую.

6. Реализация паттерна «Приспособленец»

Основная часть реализации паттерна находится в классах CellStyle и CellStyleFactory.

```
class CellStyle:
    def __init__(self, bg, fg, font, radius):
        self.bg = bg
        self.fg = fg
        self.font = font
        self.radius = radius



class CellStyleFactory:
    _cache = {}

    @classmethod
    def get(cls, key):
        if key not in cls._cache:
            if key == "normal":
                cls._cache[key] = CellStyle("#4a4a4a", "white", ("Segoe UI", 20, "bold"), 12)
            elif key == "selected":
                cls._cache[key] = CellStyle("#6a6a6a", "white", ("Segoe UI", 20, "bold"), 12)
        return cls._cache[key]

    @classmethod
    def count(cls):
```

```
        return len(cls._cache)
```

При отрисовке сетки используется один общий объект:

```
style = CellStyleFactory.get("normal")
for i in range(1, 10):
    rect = rounded_rect(self.canvas, ..., fill=style.bg, outline="")
    ...
```

При выборе ячейки стиль берётся из того же кэша:

```
normal_style = CellStyleFactory.get("normal")
selected_style = CellStyleFactory.get("selected")
for i, rect_id in self.cell_rects.items():
    style = selected_style if i in self.selected else normal_style
    self.canvas.itemconfig(rect_id, fill=style.bg)
```

Кнопка «Статистика Flyweight» показывает CellStyleFactory.count() и количество ячеек - наглядно демонстрируя, что объектов стиля в памяти меньше, чем ячеек.

7. Результаты работы программы

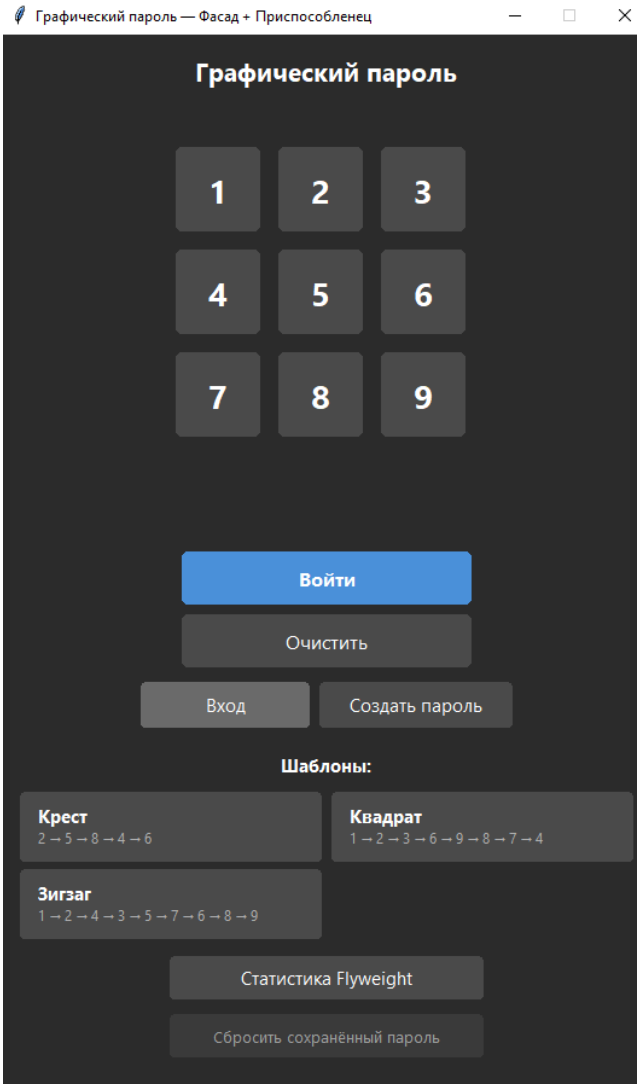


Рисунок 2 - Главное окно приложения

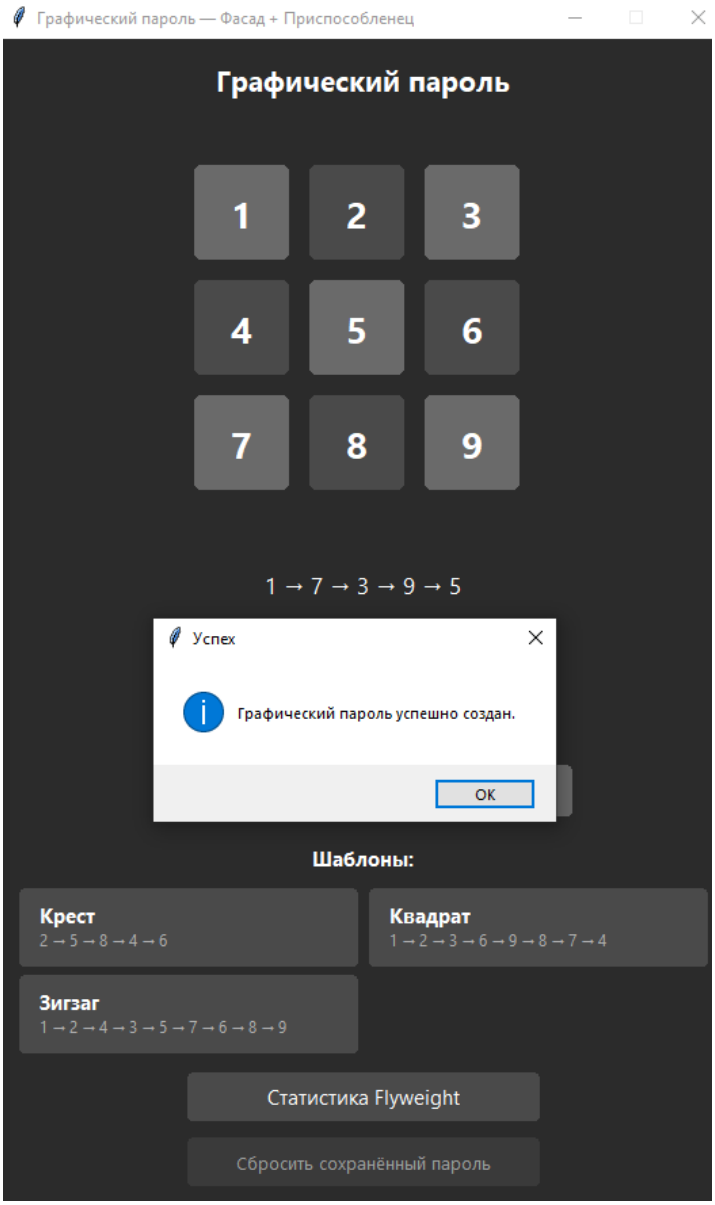


Рисунок 3 - Формирование последовательности графического пароля

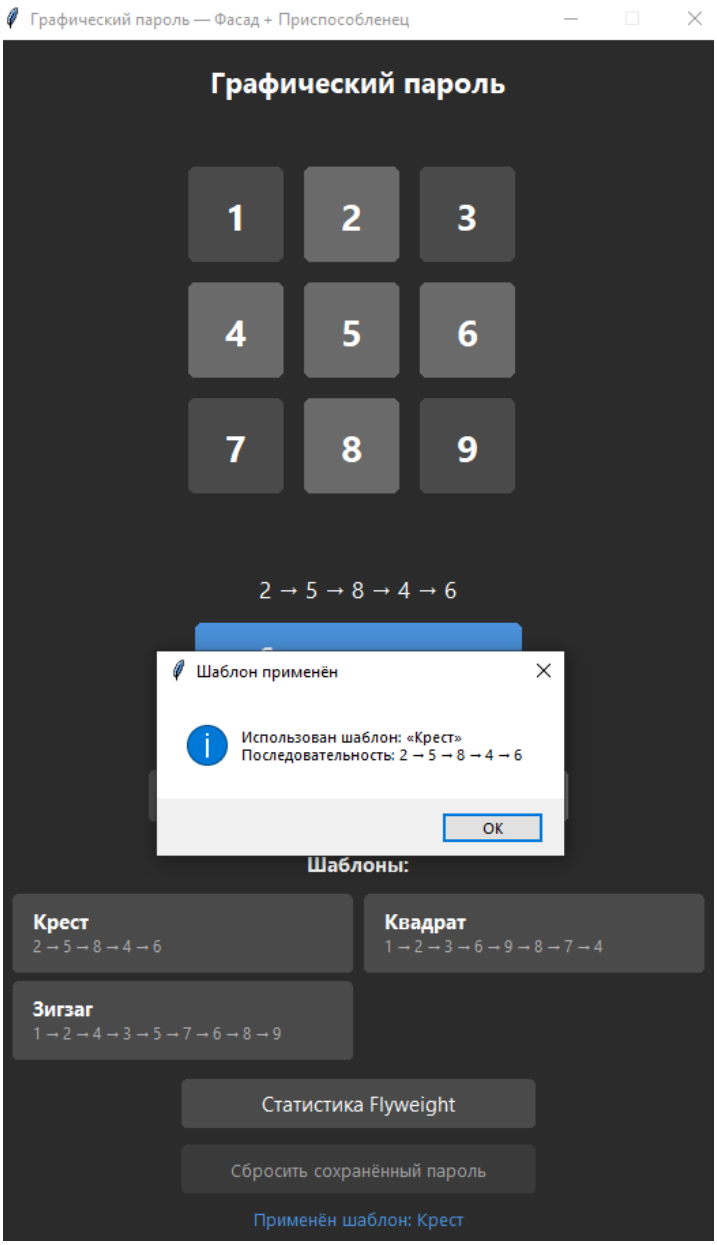


Рисунок 4 - Применение шаблона «Крест»

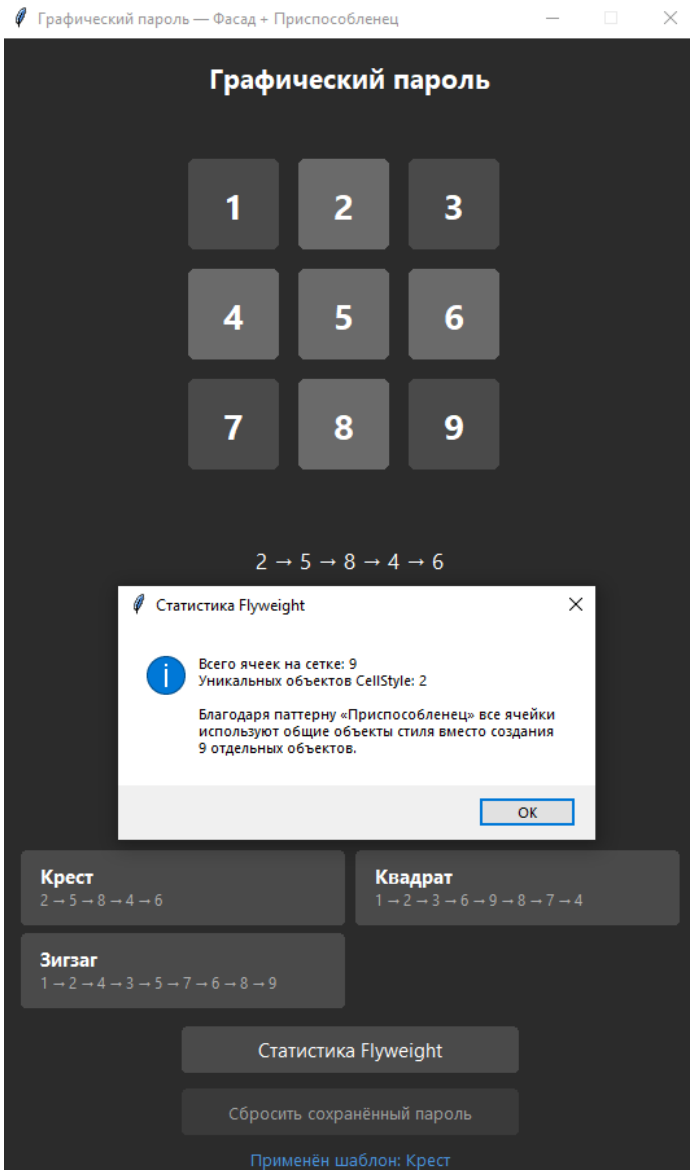


Рисунок 5 - Статистика Flyweight (9 ячеек, 2 уникальных объекта CellStyle)

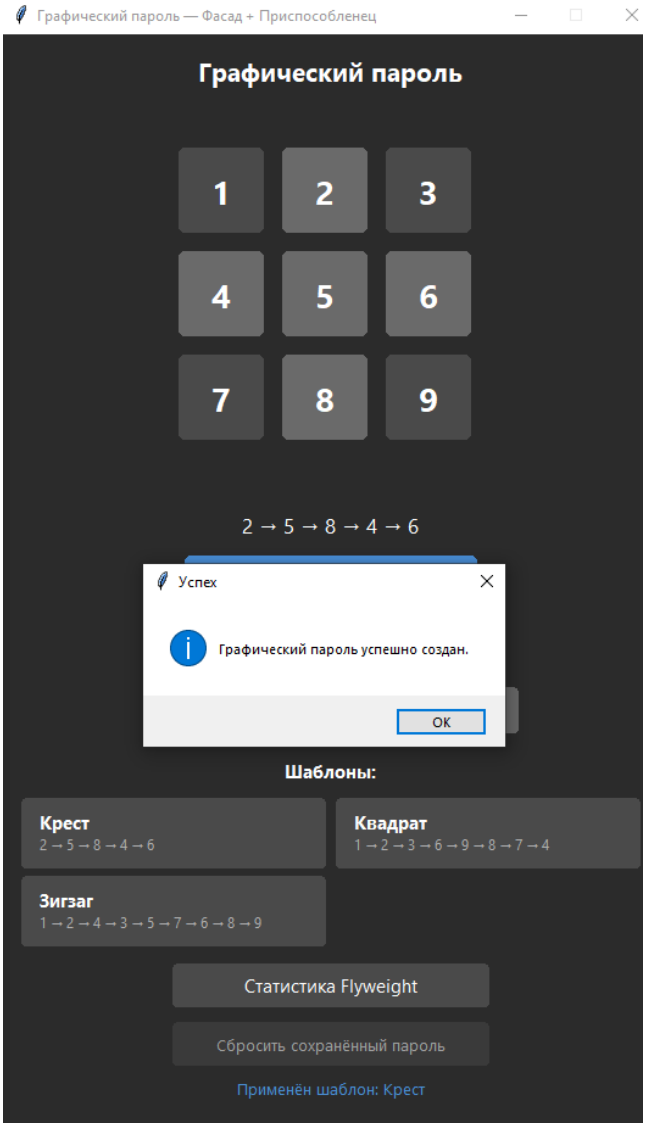


Рисунок 6 - Создание пароля

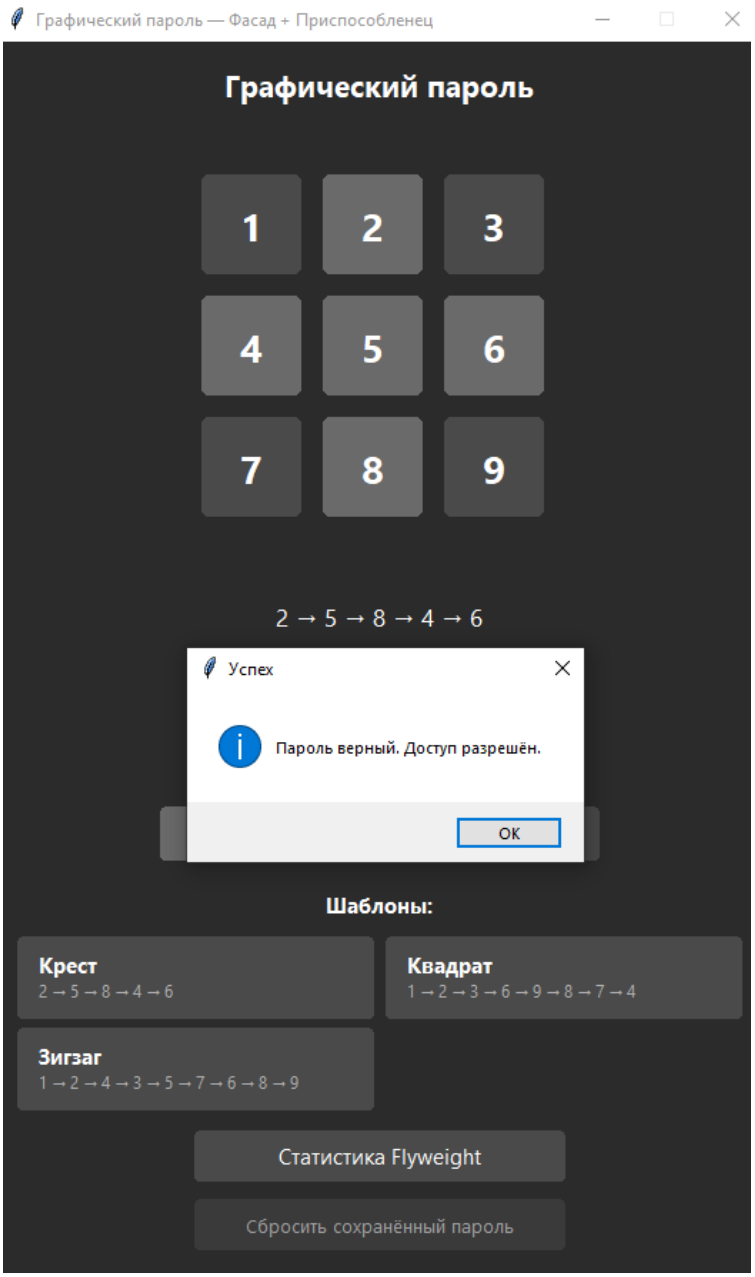


Рисунок 7 - Проверка правильного пароля

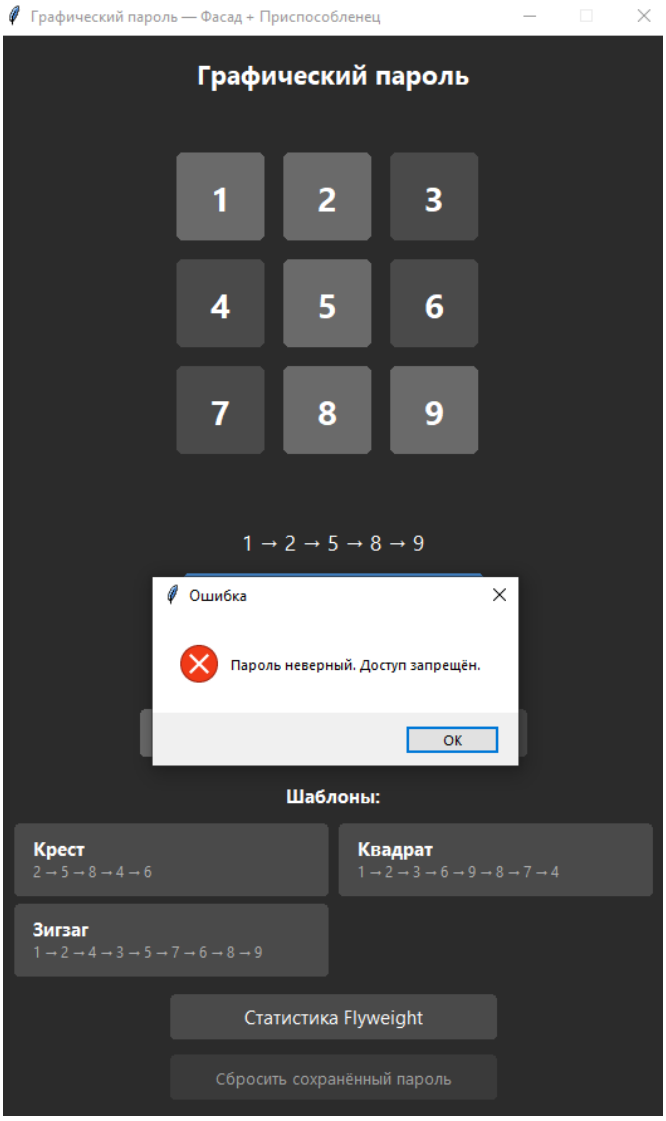


Рисунок 8 - Проверка неправильного пароля

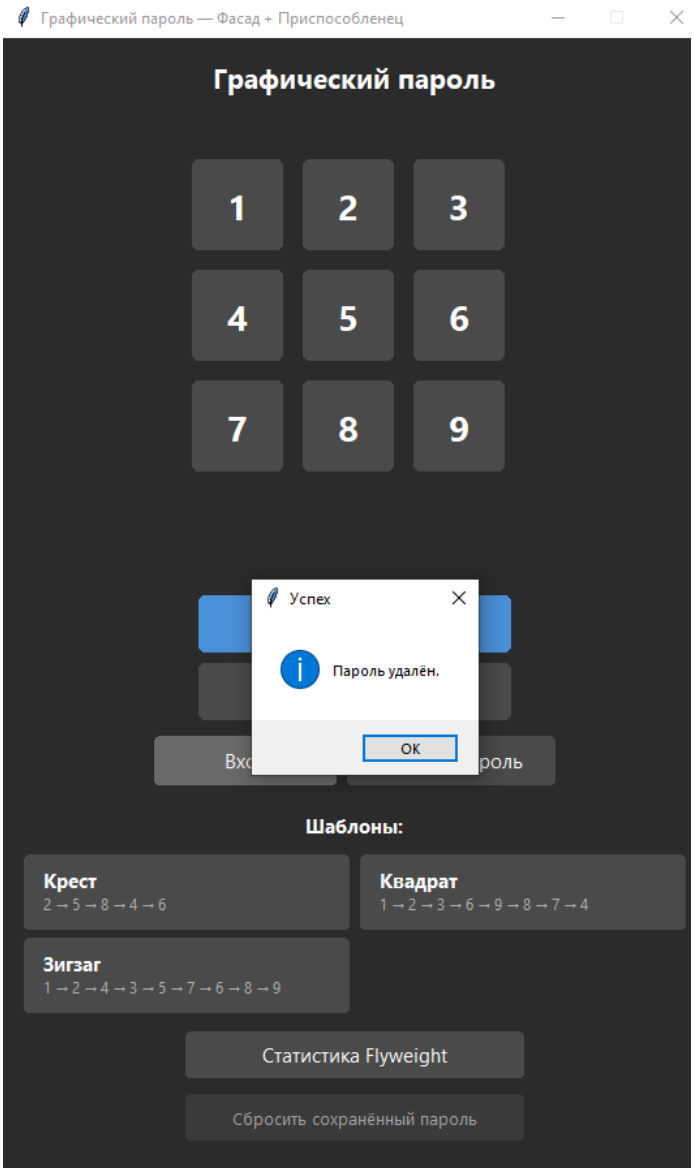


Рисунок 9 - Сброс сохранённого пароля

8. Исходный код

Основные элементы программы:

- PasswordFacade - фасад над подсистемой пароля;
- PasswordManager - Singleton с хэшированием (из лабы 1);
- PasswordPattern - шаблон графического пароля;
- CellStyle - приспособленец со стилем ячейки;
- CellStyleFactory - фабрика с кэшем (Flyweight Factory);
- AppWindow - основной класс интерфейса;
- RoundedButton, rounded_rect() - скруглённые кнопки и ячейки;
- validate_sequence(), hash_sequence() - вспомогательные функции;

– graphical_password.json - файл хранения хэша пароля.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Были изучены и реализованы два структурных паттерна проектирования: «Фасад» (Facade) и «Приспособленец» (Flyweight). Паттерн «Фасад» применён для упрощения работы с подсистемой пароля: класс PasswordFacade скрывает PasswordManager, шаблоны и валидацию за единым простым интерфейсом. Паттерн «Приспособленец» применён для экономии памяти при отрисовке ячеек: класс CellStyleFactory кэширует объекты CellStyle, и все девять ячеек используют лишь два общих объекта стиля вместо девяти отдельных. В результате получена работоспособная программа, демонстрирующая практическое применение структурных паттернов при разработке прикладного программного обеспечения.